

Modularity maximization in networks by variable neighborhood search

Daniel Aloise, Gilles Caporossi, Pierre Hansen, Leo Liberti,
Sylvain Perron, and Manuel Ruiz

ABSTRACT. Finding communities, or clusters, in networks, or graphs, has been the subject of intense studies in the last ten years. The most used criterion for that purpose, despite some recent criticism, is modularity maximization proposed by Newman and Girvan. It consists in maximizing the sum for all clusters of the number of inner edges minus the expected number of inner edges assuming the same distribution of degrees. Numerous heuristics, as well as a few exact algorithms have been proposed to maximize modularity. We apply the Variable Neighborhood Search metaheuristic to that problem. Computational results are reported for the instances of the 10th DIMACS Implementation Challenge. The algorithm presented in this paper obtained the second prize in the quality modularity (sub)challenge of the referred competition, finding the best known solutions for 11 out of 30 instances.

Data
used

See overview of the example

1. Introduction

Clustering is an important chapter of data analysis and data mining with numerous applications in natural and social sciences as well as in engineering and medicine. It aims at solving the following general problem: given a set of entities, find subsets, or clusters, which are homogeneous and/or well-separated. As the concepts of homogeneity and of separation can be made precise in many ways, there are a large variety of clustering problems [HJ, JMF, KR, M]. These problems in turn are solved by exact algorithms or, more often and particularly for large data sets, by heuristics, of which there are frequently a large variety. An exact algorithm provides, hopefully in reasonable computing time, an optimal solution together with a

1991 *Mathematics Subject Classification.* Primary 90C27, 90C59, 91C20.

Key words and phrases. Modularity, community, clustering.

The first author was partially supported by the National Council for Scientific and Technological Development - CNPq/Brazil grant number 305070/2011-8.

The second author was partially supported by FQRNT (Fonds de recherche du Québec - Nature et technologies) team grant PR-131365 and NSERC (Natural Sciences and Engineering Research Council of Canada) grant 298138-2009.

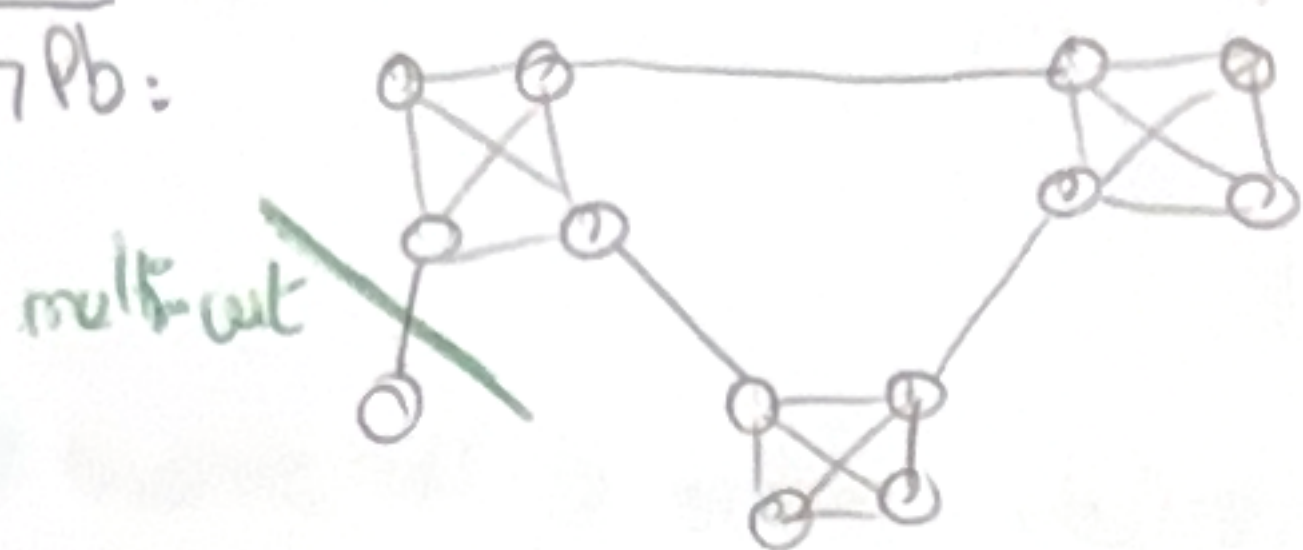
The third author was partially supported by FQRNT team grant PR-131365, NSERC grant 105574-07 and Digitco grant Senior Chair 2009-14D "RMNCCO".

The fourth author was partially supported by Digitco grant Senior Chair 2009-14D "RMNCCO".

The fifth author was partially supported by FQRNT team grant PR-131365 and NSERC grant 327435-06.

- Multway cut: Minimize the total number of edges that cross cluster boundaries: 3 here.

↳ Pb:



~ will found 2 clusters (obj. fun = 1)

But the good answer is 3 clusters

Solu 0 = 7 Ratio Cut = penalize clusters that have too

Score = $\frac{\sum \text{cut size}}{\sum \text{Nodes}}$ few nodes.

$$\text{Score}_{3\text{clust}} = \frac{2}{4} + \frac{2}{4} + \frac{2}{5} = 1.4 \checkmark$$

$$\text{Score}_{2\text{clust}} = \frac{1}{1} + \frac{1}{12} = 1.08?$$

114 D. ALOISE, G. CAPOROSSI, P. HANSEN, L. LIBERTI, S. PERRON, AND M. RUIZ

proof of its optimality. A heuristic provides, usually in moderate computing time, a near optimal solution or sometimes an optimal solution but without proof of its optimality.

In the last decade, clustering on networks, or graphs, has been extensively studied, mostly in the physics and computer science communities, with recently a few forays from operations research. Rather than using the term cluster, the words *module* or *community* are often adopted in the physics literature. We use below the standard notation and terminology for graphs, i.e. a graph $G = (V, E, \omega)$ is composed of a set V of n vertices v_j and a set E of m edges $e_{ij} = \{v_i, v_j\}$. These edges may be weighted by the function $\omega(\{u, v\})$. If they are unweighted $\omega(\{u, v\}) = 1$. A subgraph $G_C = (C, E_C, \omega)$ of a graph $G = (V, E, \omega)$ induced by a set of vertices $C \subseteq V$ is a graph with vertex set C and edge set E_C equal to all edges with both vertices in C . Such a subgraph corresponds to a **cluster** (or module, or community) and many heuristics aim at finding a partition C of V into pairwise disjoint nonempty subsets $V_1, V_2, \dots, V_N$ inducing subgraphs of G and covering V . **Various objective functions have been proposed for evaluating such a partition.** Among the best known are **multway cut** [GH], **normalized cut** [SM], **ratio cut** [AY] and **modularity** [NG]. [Initially proposed by Girvan and Newman in 2002 [GN] as a stopping rule for a hierarchical divisive heuristic, **modularity** was considered later as an independent criterion allowing determination of optimal partitions as well as comparison between partitions obtained by various methods.]

Modularity aims at finding a partition of V which maximizes the sum, over all modules, of the number of inner edges minus the expected number of such edges **assuming that they are drawn at random with the same distribution of degrees as in G .** The following precise definition of **modularity** is given in [NG]:

$$Q = \sum_{C \in \mathcal{C}} [a_C - e_C] = \text{Modularity of a Cluster.}$$

e_C expected cluster

where a_C is the fraction of all edges that lie within module C and e_C is the expected value of the same quantity in a graph in which the vertices have the same expected degrees but edges are placed at **random**. A maximum value of Q near to 0 indicates that the network considered is close to a random one (barring fluctuations), while a maximum value of Q near to 1 indicates strong community structure. Observe that maximizing modularity gives an optimal partition together with the optimal number of modules.

Let the weight vertex function be defined as:

$$\omega(v) = \begin{cases} \sum_{\{u,v\} \in E} \omega(\{u,v\}) & \text{if } \{v,v\} \notin E \\ \sum_{\{u,v\} \in E, u \neq v} \omega(\{u,v\}) + \frac{2\omega(\{v,v\})}{2} & \text{if } \{v,v\} \in E. \end{cases}$$

Self-loop

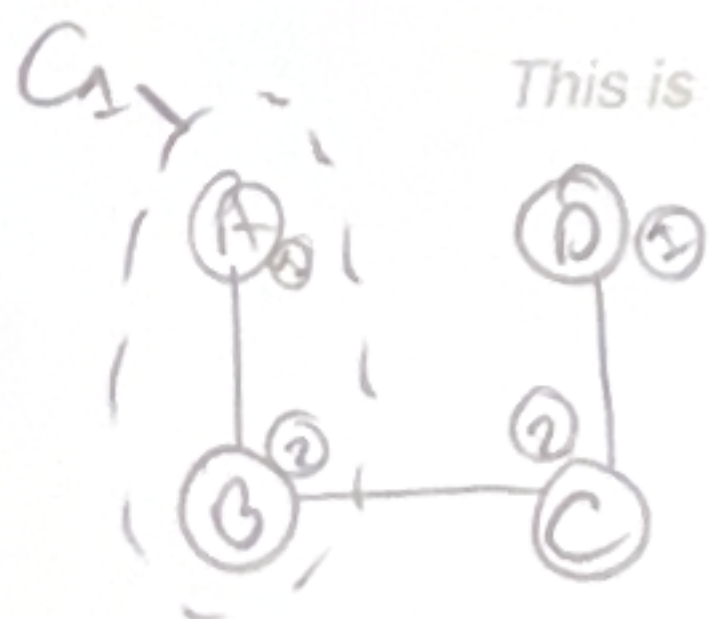
The vertex has a self-loop.

The modularity for a module C may be written as

$$(1) \quad Q(C) = \frac{\sum_{\{u,v\} \in E_C} \omega(\{u,v\})}{\sum_{e \in E} \omega(e)} - \frac{\left(\sum_{v \in V_C} \omega(v) \right)^2}{4 \left(\sum_{e \in E} \omega(e) \right)^2}$$

Modularity of a Cluster

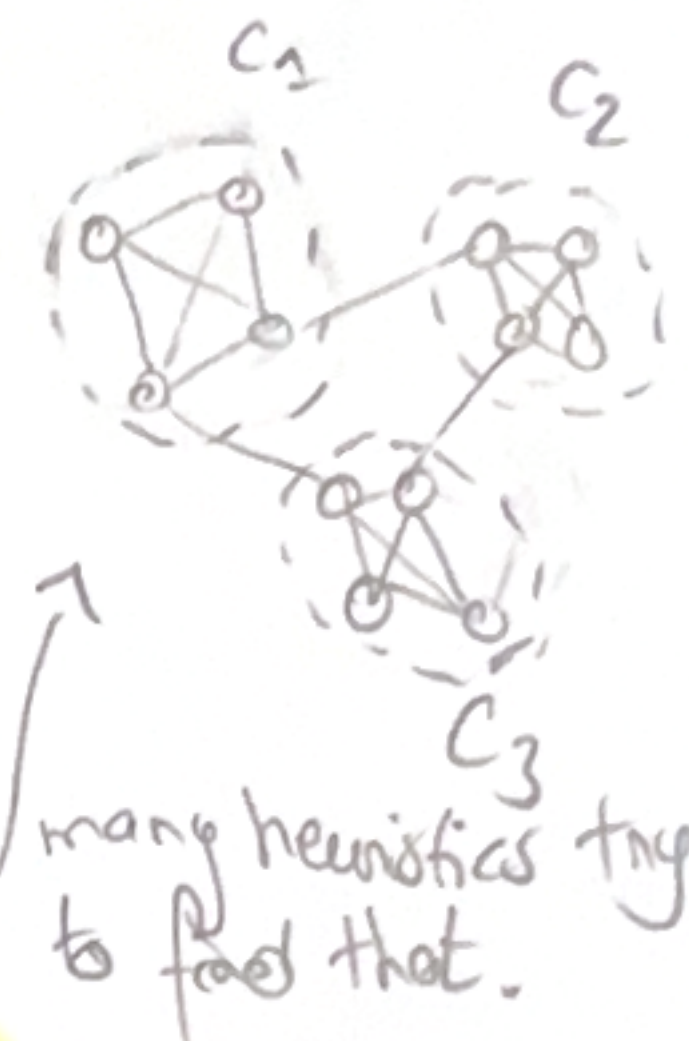
$2 \sum_{e \in E} \omega(e) = \sum \text{degrees in the graph}$



$$Q(C_1) = \frac{4}{3} - \left(\frac{3}{6} \times \frac{3}{6} \right) = \frac{4}{3} - 0.25 = 0.083.$$

To compute e_{C_1} , we cut all edges, throw the 6 stubs into a bag, and tie them together blindly. e_C = probab of creating an edge entirely inside C_1 →

2/12



Let $\mathcal{C}$ be a partition of V . The sum over modules of their modularities can be written as

$$(2) \quad Q = \frac{\sum_{C \in \mathcal{C}} \sum_{\{u,v\} \in E_C} \omega(\{u,v\})}{\sum_{e \in E} \omega(e)} - \frac{\sum_{C \in \mathcal{C}} \left(\sum_{v \in V_C} \omega(v) \right)^2}{4 \left(\sum_{e \in E} \omega(e) \right)^2} \quad \left. \vphantom{\sum_{C \in \mathcal{C}}} \right\} \text{Modularity of the Network.}$$

Numerous heuristics have been proposed to maximize modularity. They are based on divisive hierarchical clustering, agglomerative hierarchical clustering, partitioning, and hybrids. They rely upon various criteria for agglomeration or division [BGLL, CNM, DDA, N04, WT], simulated annealing [GA, MD, MAD], mean field annealing [LH], genetic search [THB], extremal optimization [DA], label propagation [BC, LM], spectral clustering [N06, RMP, SDJB], linear programming followed by randomized rounding [AK], dynamical clustering [BILPR], multilevel partitioning [D], contraction-dilation [MHSWL], multistep greedy search [SC], quantum mechanics [NHZW] and other approaches [BGLL, CFS, FLZWD, KSKK, RZ, SDJB]. For a more detailed survey, see [F]. While other metaheuristics have been applied to modularity maximization, it is the first time, to the best of our knowledge, that the **Variable Neighborhood Search (VNS)** metaheuristic is used for that purpose. In particular, by using decomposition, we were able to tackle larger problems than the previous metaheuristic approaches, reducing the size of the problem in which VNS is applied.

The paper is organized as follows: in Section 2, after giving an outline of the VNS metaheuristic, we discuss its application to modularity maximization. In Section 3, we recall and extend to the weighted case an exact method for modularity maximization which is used to evaluate the quality of the solutions obtained by our variable neighborhood search metaheuristic. Experimental results are presented in Section 4 in two tables corresponding to the results for Pareto and Quality challenges respectively. Brief conclusions are drawn in the last section.

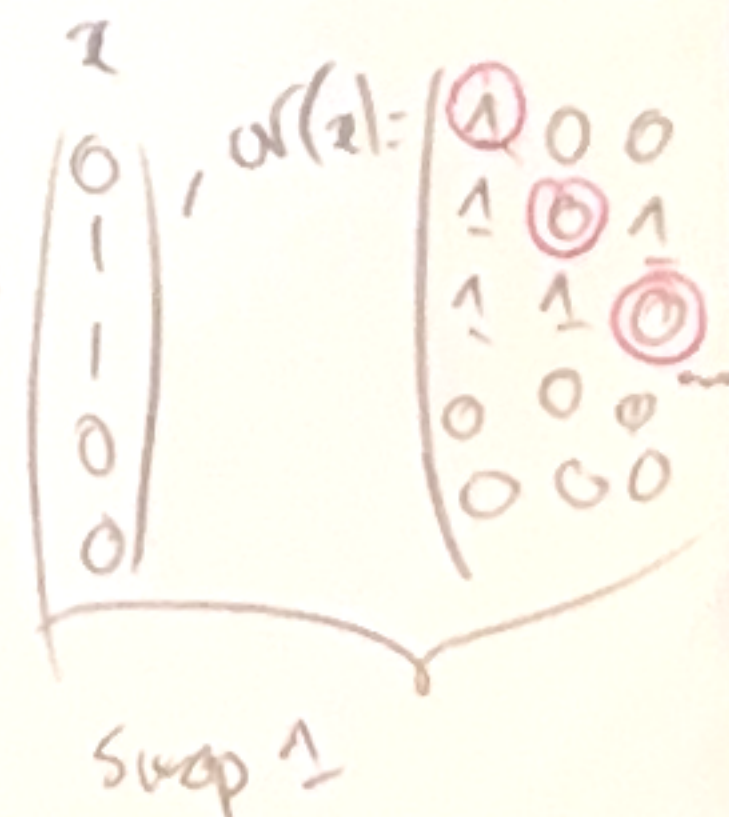
2. Description of the heuristic

2.1. Outline of the variable neighborhood search metaheuristic. Variable Neighborhood Search (VNS) is a metaheuristic, or framework for building heuristics, aimed at solving combinatorial and global optimization problems. Since its inception, VNS has undergone many developments and has been applied in numerous fields (see [HMP] for a recent survey).

Metaheuristics address the problem of escaping, as much as possible, from local optima. A local maximum x_L of an optimization problem is such that

$$(3) \quad f(x_L) \geq f(x), \forall x \in N(x_L)$$

where $N(x)$ denotes the feasible neighborhood of x , which can be defined in many different ways each one yielding a different neighborhood structure. In discrete optimization problems, a neighborhood structure consists of all vectors obtained from x by some simple modification. For instance, for x binary, one neighborhood structure can be defined by the set of all vectors obtained from x by complementing one of its components. Another possible neighborhood structure can be defined as the set of all vectors obtained from x by complementing two complementary



Swap 2: $\begin{pmatrix} 1 \\ 0 \\ 1 \\ 0 \\ 0 \end{pmatrix}, w(x) = \begin{pmatrix} 1 & 1 & 1 & 1 & \dots \\ 0 & 1 & 1 & 1 & \dots \\ 1 & 0 & 1 & 1 & \dots \\ 0 & 0 & 1 & 1 & \dots \\ 0 & 0 & 1 & 1 & \dots \end{pmatrix}$

components of x (i.e., one component is set from 0 to 1 and the other goes from 1 to 0). A **local search** or **improving** heuristic consists of choosing an initial solution x , and then moving to the best neighbor $x' \in N(x)$ in the case $f(x') > f(x)$. If no such neighbor exists, the heuristic stops, otherwise it is iterated.

If many local maxima exist for a problem, the range of values they span may be large. Moreover, the globally optimum value $f(x^*)$ may differ substantially from the average value of a local maximum, or even from the best such value among many, obtained by some simple randomized heuristic. In order to escape from local maxima and, more precisely, the mountains of which they are the top, **VNS exploits the idea of neighborhood change**. In fact, VNS relies upon the following observations:

Fact 1: A local maximum with respect to one neighborhood structure is not necessarily so for another; (Swap 1 and Swap 2 $\Rightarrow$ we don't obtain the same local max).

Fact 2: A global maximum is a local maximum with respect to all possible neighborhood structures;

Fact 3: For many problems local maxima with respect to one or several neighborhoods are relatively close to each other.

Let us denote with $\mathcal{N}_t$, ($t = 1, \dots, t_{max}$), a finite set of pre-selected neighborhood structures, and with $\mathcal{N}_t(x)$ the set of solutions in the t^{th} neighborhood of x . We call x a **local maximum** with respect to $\mathcal{N}_t$ if there is no solution $x' \in \mathcal{N}_t(x)$ such that $f(x') > f(x)$.

In the VNS framework, the neighborhoods used correspond to various types of moves, or perturbations, of the current solution, and are **problem specific**. The current best solution x found is the center of the search. When looking for a better one, a solution x' is drawn at random in an increasingly far neighborhood and a local ascent is performed from x' , leading to another local maximum x'' . If $f(x'') \leq f(x)$, x'' is ignored and one chooses a new neighbor solution x' in a further neighborhood of x . If, otherwise, $f(x'') > f(x)$, the search is re-centered around x'' restarting with the closest neighborhood. If all neighborhoods of x have been explored without success, one begins again with the closest one to x , until a stopping condition (e.g. maximum CPU time) is satisfied.

As the size of neighborhoods tends to increase with their distance from the current best solution x , close-by neighborhoods are explored more thoroughly than far away ones. This strategy takes advantage of the three Facts 1–3 mentioned above. Indeed it is often observed that most or all local maxima of combinatorial problems are concentrated in a small part of the solution space. Thus, finding a first local maximum x implies that some important information has been obtained: to get a better, near-optimal solution, one should first explore its vicinity.

The algorithm proposed in this work has two main components: (i) an **improvement heuristic**, and (ii) **exploration of different types of neighborhoods for getting out of local maxima**. They are used within a variable neighborhood decomposition search framework [HMP] which explores the structure of the problem concentrating on small parts of it. The basic components as well as the decomposition framework are described in the next sections.

2.2. Improvement heuristic. The improvement heuristic we used is the

LPAm+ algorithm proposed by Liu and Murata in [LM]. LPAm+ is composed of a **label propagation algorithm** proposed by Barber and Clark [BC] and a **community merging routine**. A strong feature of this heuristic is that label propagation

First, understand
① LPAm (with modularity maxing)
② LPAm+ (with community merging)

This is a free offprint provided to the author by the publisher. Copyright restrictions may apply.

Example of LPA algo overlap.

Ex. Start 1 = Swap 1
Start 2 = Swap 2
...



distance -- = swap 1
distance ++ = swap 20.

$$LPA_{m+} \mathcal{C}: \underbrace{O(m)}_{\text{label propagation}} + \underbrace{O(m \log n)}_{\text{community merging}} = \boxed{O(m \log n)}$$

1 part article → LPA_m favors community divisions where all communities are similar in total deg.
 ⇒ Can lead to local max
 ⇒ Community Merging (LPA_{m+})

executes in near **linear time** (in fact, each iteration of label propagation executes in time proportional to m), while one round of merging pairs of communities can execute in $O(m \log n)$ [SC].

Label propagation is a similarity-based technique in which the label of a vertex is propagated to adjacent vertices according to their proximity. Label propagation algorithms for clustering problems assume that the label of a node correspond to its incumbent cluster index. Then, at each label propagation step, each vertex is sequentially evaluated for label updating according to a propagation rule. In [BC], the Barber and Clark propose a label propagation algorithm, called LPA, for modularity maximization. Their label updating rule for vertex v is (see. [BC] for details):

$$(4) \quad \ell_v \leftarrow \underset{\ell}{\operatorname{argmax}} \left(\sum_{u=1}^n B_{uv} \delta(\ell_u, \ell) \right)$$

where ℓ_v is the label of vertex v , $B_{uv} = \omega(\{u, v\}) - (\omega(u)\omega(v))/2m$, and $\delta(i, j)$ is the Kronecker's delta.

Moreover, the authors prove that the candidate labels for ℓ_v in eq.(4) can be confined to the labels of the vertices adjacent to v and an unused label. We decided to use this fact in order to speedup LPA. Let us consider a vertex $v^* \in C$, where C is a module of the current partition, and let us suppose that the modules to which its adjacent vertices belong have not changed since the last evaluation of v^* . In this case, v^* can be discarded for evaluation since no value has changed from the last instantiation of eq.(4) since the last evaluation of v^* . With that in mind, **we decided to iterate over "labels" instead of over the vertices of the graph.**

We used LPA_{m+} modified as follows. A list L of all labels is initialized with the clusters indices of the current partition. Then, from L , we proceed by picking a label $\ell \in L$ until L is empty. Each time a label ℓ is removed from L , we evaluate by means of eq.(4) all its vertices for label updating. If the label of a vertex is updated, yielding an improvement in the modularity value of the current partition, the old and the new labels of that vertex, denoted ℓ^{old} and ℓ^{new} , are inserted in L . Moreover, the labels of vertices which are connected to a node with label equal to either ℓ^{old} or ℓ^{new} are also inserted in L . This modification induces a considerable algorithmic speedup since only a few labels need to be evaluated as the algorithm proceeds.

We then tested this modified LPA_{m+}, and proceeded to improve it based on empirical observations. In the final version, whenever a vertex relabeling yields an improvement, the old and the new labels of that vertex are added to L but only together with the labels of vertices which are adjacent to the relabeled vertex. This version was selected to be used in our experiments due to its benefits in terms of computing times and modularity maximization.

2.3. Neighborhoods for perturbations. In order to escape from local maxima, our algorithm uses five distinct neighborhoods for perturbing a solution. They are:

- (1) SINGLETON: all the vertices in a cluster are made singleton clusters.
- (2) DIVISION: splits a community into two equal parts. Vertices are assigned to each part randomly.

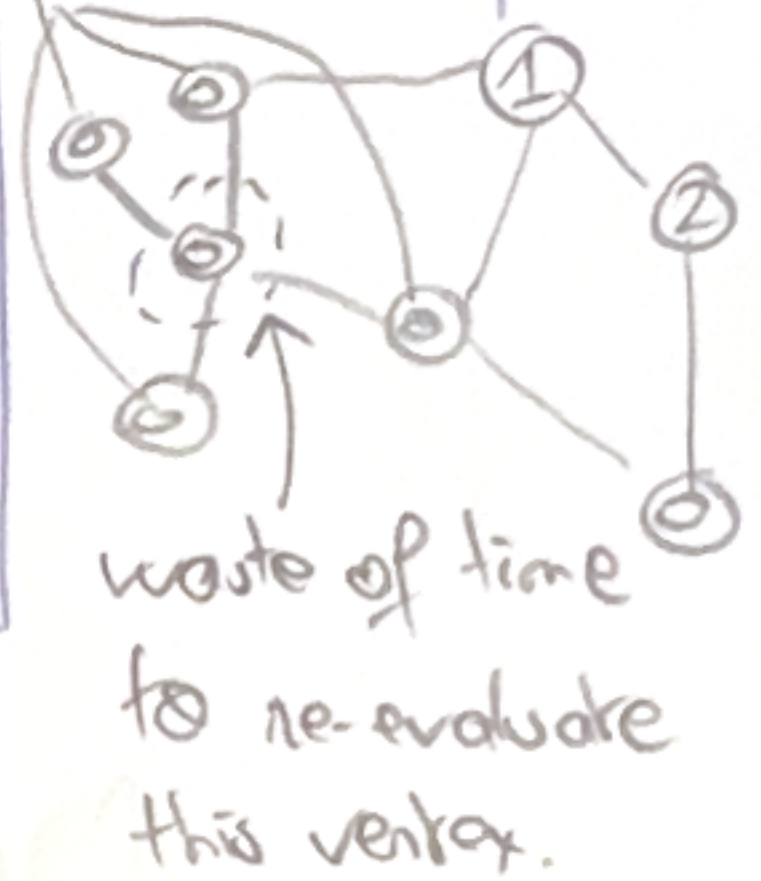
LPA_m fast.
 implemented ✓

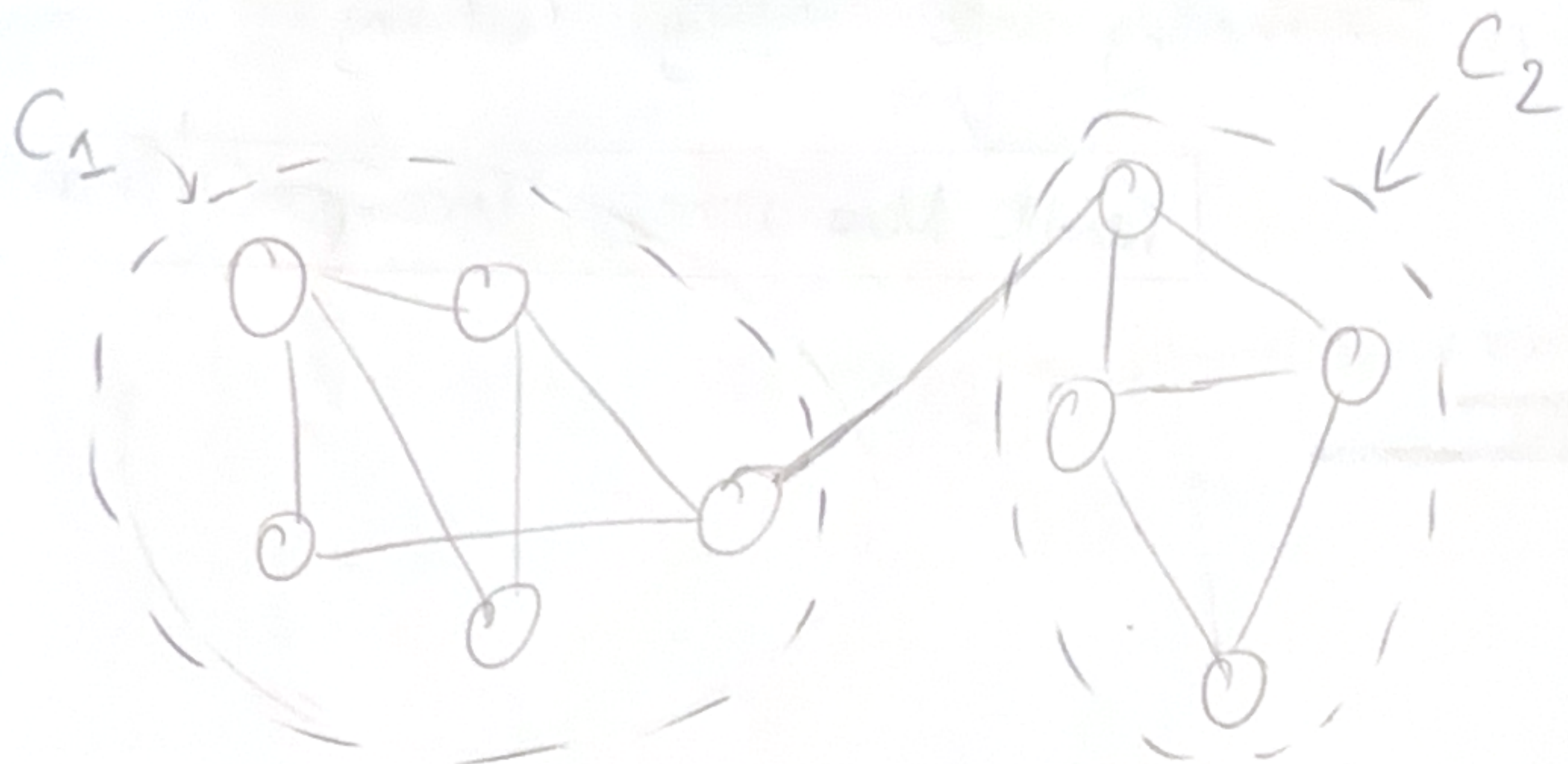
LPA_{m+} fast.
 (not implemented) ×
 Yet: 29/07/201

implemented in
 the nb.

didn't change

→ Add proof nb.





$$C_1\text{-inner edges} = 6$$

$$C_1\text{-exp-inner edges} = e_1$$

$$C_1\text{-mod-contrib} = 6 - e_1$$

$$C_2\text{-inner edges} = 5$$

$$C_2\text{-exp-inner edges} = e_2$$

$$C_2\text{-mod-contrib} = 5 - e_2$$

Total $\max \sum_i C_i\text{-mod-contrib} = \max (6 - e_1 + 5 - e_2)$
 $(1) \begin{cases} e_1 = 6 \\ e_2 = 5 \end{cases} \text{ here}$

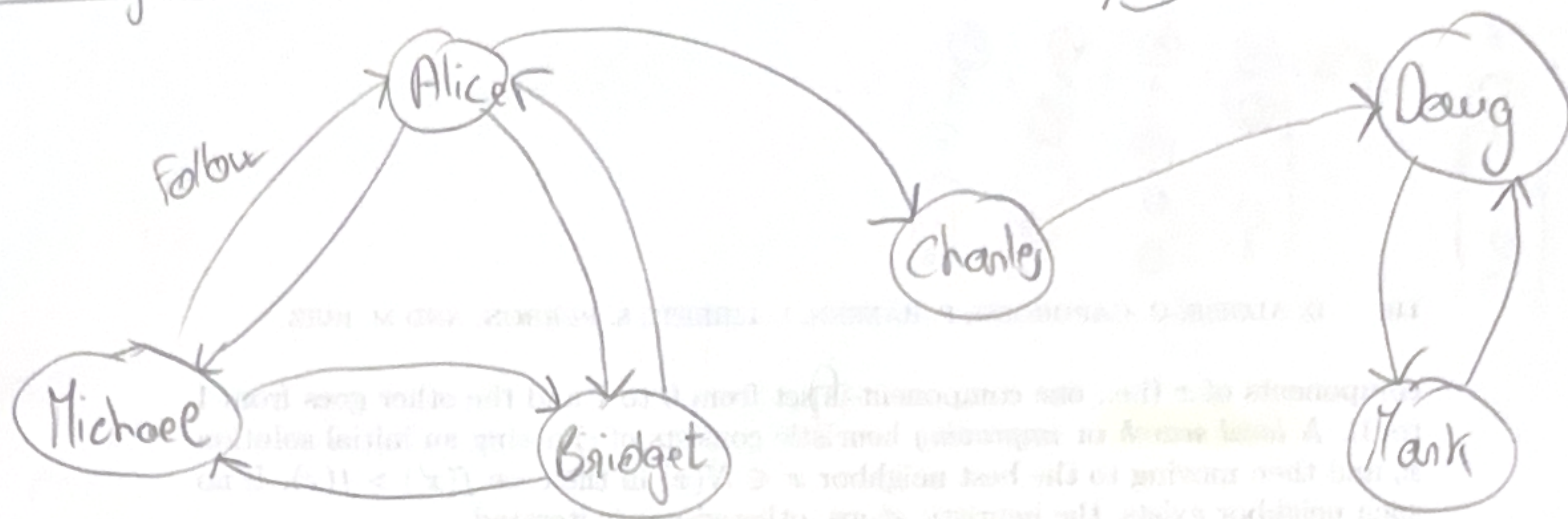
$$\text{Modularity of } C_1: Q_{C_1} = \frac{6}{12}$$

Simple LPA algorithm:

Node = {name, seed-label} can be the result of a previous LPA

n=6
m=10

Step 0



Step 1



- Each node is assigned with a unique label.
- At every label propagation step, each node sequentially updates its label to a new one which is the most frequent label among its neighbors.

$$A = \begin{pmatrix} 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{pmatrix}$$

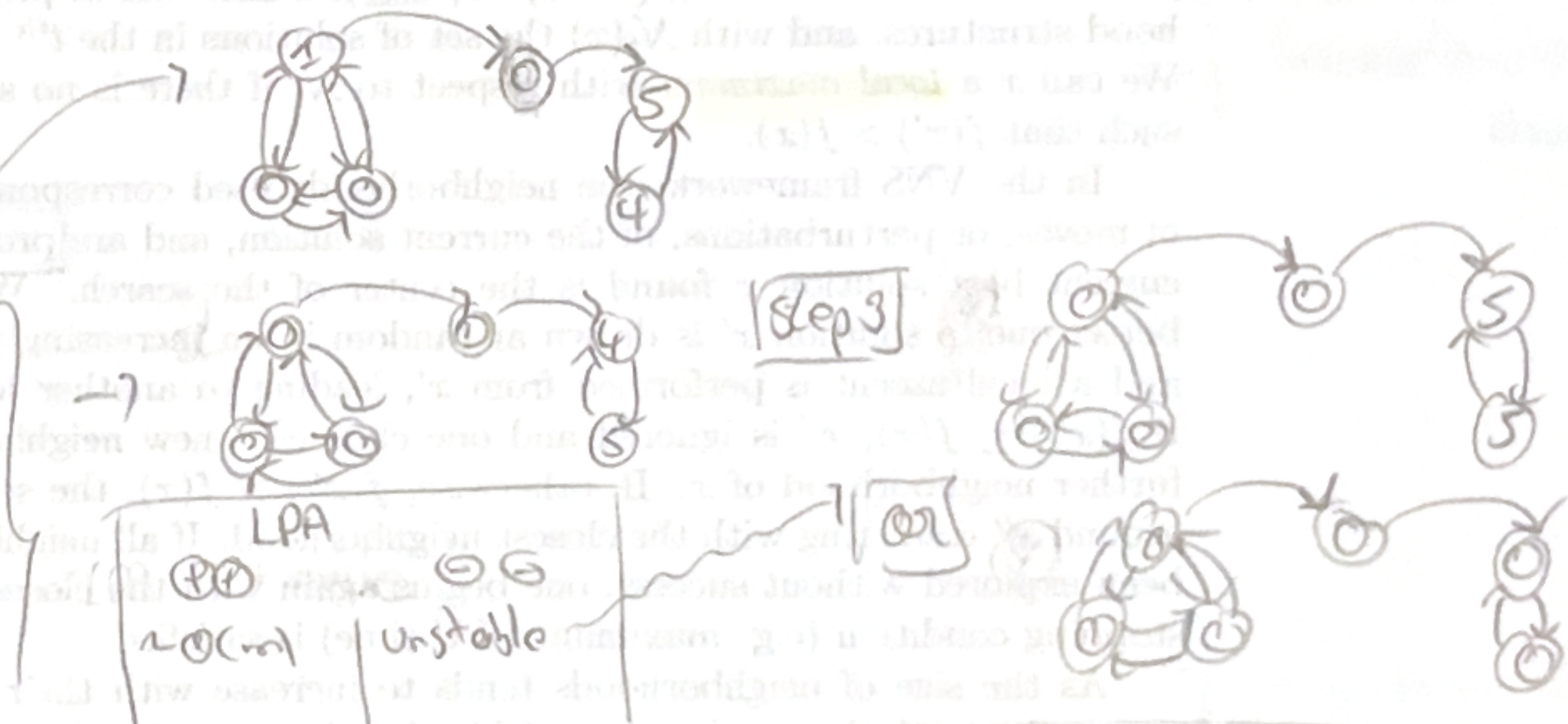
$L_x^{new} = 1$ (equal = random)
 $L_0^{new} = 0$
 $L_1^{new} = 0$
 $L_2^{new} = 0$
 $L_3^{new} = 0$
 $L_4^{new} = 5$
 $L_5^{new} = 4$

$$LPA: L_x^{new} = \arg \max_L \sum_{u \sim x} A_{ux} S(e_u, e_x)$$

Adjacency matrix filter the current label.

Step 2

$L_0^{new} = 0$
 $L_1^{new} = 0$
 $L_2^{new} = 0$
 $L_3^{new} = 0$
 $L_4^{new} = 4$
 $L_5^{new} = 5$



LPA m Algo

→ LPA with modularity maximization in each cluster.

$$L_x^{new} = \arg \max_L \left(\sum_{u \sim x} B_{ux} S(e_u, e_x) \right)$$

$$B_{ux} = w(\{u, x\}) - \frac{w(u)w(x)}{2m} \in (-1, 1]$$

modularity matrix

A_{ux}

a random network.

Prober that an edge would exist between u and x if all the connections in the network were made randomly.